Supply Chain Intelligence System: Project Report

Oct 18, 2026 · @mayank

1. Executive Summary

A Random Forest model with 74% accuracy now flags orders at risk of late delivery before they ship, aimed at a 54.70% late rate that puts $2.1M of profit at risk.

The Supply Chain Intelligence System analyzes 172,000+ historical orders to find where and when deliveries fail, then turns that analysis into two tools: a performance dashboard for operations leaders and a prediction engine for planners and fulfillment teams. Both run in a Streamlit web app.

The central finding is that lateness is the norm, not the exception. More than half of orders arrive late, yet the 90th percentile delay is only 3 days. Delays are small, frequent and tied to process, which makes them fixable.

Headline metric	Value
Historical orders analyzed	172,000+
Late delivery rate	54.70%
On-time delivery rate	45.30%
Profit at risk (delayed orders)	$2.1M
Total profit (profitable orders)	$7.9M
Average order profit	$21.97
90th percentile delay	3 days
Model accuracy (34,553 test orders)	74%
Precision on late orders	79%
Recall on late orders	75%

2. Business Problem & Objectives

Late deliveries are a margin problem, a customer-experience problem and a planning problem at once, and today most teams only learn an order is late after it is.

In e-commerce, retail and third-party logistics, delivery promises are the product. A missed promise triggers refunds, support contacts, churn and penalties, and it erodes the profit the order was meant to earn. Here, orders that arrived late carry $2.1M of profit that is under pressure.

Reporting tools describe what already happened. Planners need the opposite: an early signal at the moment an order is placed, while there is still time to change the shipping mode, reroute stock or reset the customer's expectation.

Project objectives

1. Quantify the delay problem: late rate, profit exposure and delay severity.
2. Locate the bottlenecks by shipping mode, region, segment and month.
3. Predict at-risk orders using only information known at order time.
4. Deliver both views in one app that non-technical users can operate.

3. Data & Scope

The model predicts one binary outcome, `is_delayed`, from nine order-time inputs drawn from the historical order file `SupplyChainData.csv`.

The dashboard reports 180,519 orders analyzed, while the modeling work uses 172,000+ orders, of which 34,553 were held out for testing; state both figures wherever the order count is published.

Input feature	Type	Why it matters
Transaction Type	Categorical	Payment method can affect order release time
Customer Segment	Categorical	Segments may have different service levels
Order Region	Categorical	Geography drives lead time and carrier mix

Input feature	Type	Why it matters
Category Name	Categorical, high cardinality	Product mix affects handling and stock
Department Name	Categorical	Groups products by fulfillment flow
Shipping Mode	Categorical	The strongest bottleneck signal in the dashboard
Scheduled Shipping Days	Numeric	The promise the order is measured against
Order Month	Numeric (1-12)	Captures seasonality
Order Hour	Numeric (0-23)	Captures cut-off and queue timing

Data leakage controls. Fields only known after fulfillment, such as actual shipping days and delivery status, were removed. They would reveal the outcome and inflate accuracy, but they cannot exist when a new order is scored.

Scope. The system covers prediction at order placement and descriptive analytics on past orders. It does not yet model carrier-level events, weather, inventory or live tracking.

4. Methodology

The workflow has five stages, each chosen so the model is trustworthy in production, not only accurate on paper.

1. **Data cleaning and leakage removal.** Columns that encode the outcome, including actual shipping days and delivery status, were dropped. The model sees only what a planner would know when the order is placed.
2. **Feature engineering with frequency encoding.** High-cardinality categories such as product category were encoded by how often each value appears. This keeps the feature count small, avoids the width of one-hot encoding and lets a tree model use how common a category is.
3. **Class balancing with SMOTE.** The target `is_delayed` was balanced with the Synthetic Minority Over-sampling Technique, so the model does not simply favor the majority class.

4. **Model tuning.** A Random Forest Classifier was tuned with `RandomizedSearchCV`, which samples the hyperparameter space instead of exhausting it and keeps training time practical.
5. **Serialization and deployment.** The tuned model was saved as `rf_model_balanced.pkl` and loaded by the Streamlit app, so predictions run on the same encoding and features used in training.

Why a Random Forest. It handles mixed categorical and numeric inputs, tolerates non-linear interactions such as shipping mode combined with month, resists overfitting through averaging, and exposes feature importance for later explanation work.

5. Model Performance & Evaluation

The tuned Random Forest reaches 74% accuracy on 34,553 held-out orders, with 79% precision and 75% recall on late deliveries.

The test set is 57.3% late (19,795 of 34,553 orders), so a rule that labels every order late would score 57.3%. The model's 74% accuracy is about 17 points above that baseline, and it was measured on real orders that SMOTE never touched.

The app returns a probability, not just a label. In the example shown, an order with a 48.0% delay probability is classed as likely on time, which implies a 50% decision threshold.

Class	Precision	Recall	F1 score	Orders (support)
On time (0)	0.68	0.73	0.70	14,758
Late (1)	0.79	0.75	0.77	19,795
Macro average	0.73	0.74	0.73	34,553
Weighted average	0.74	0.74	0.74	34,553

Reading the late class. Of every 100 orders the model flags as late, about 79 really are late, so roughly one flag in five is a false alarm. Of every 100 orders that really are late, about 75 are caught, so roughly one in four slips through.

The on-time class is weaker. At 0.68 precision, about a third of orders predicted on time turn out late. Use the model to prioritize attention, not to guarantee an on-time promise.

Still to report. ROC-AUC, a confusion matrix in order counts and cross-validated scores would complete the evaluation and show how stable the 74% is across data splits.

Threshold choice is a business decision. Lowering the threshold catches more late orders at the cost of more false alarms. Teams should set it from the relative cost of an expedited shipment versus a missed delivery, not leave it at 50%.

6. Dashboard & Application Design

One Streamlit app serves two audiences through a sidebar switch: managers who need to see the problem, and planners who need to act on a single order.

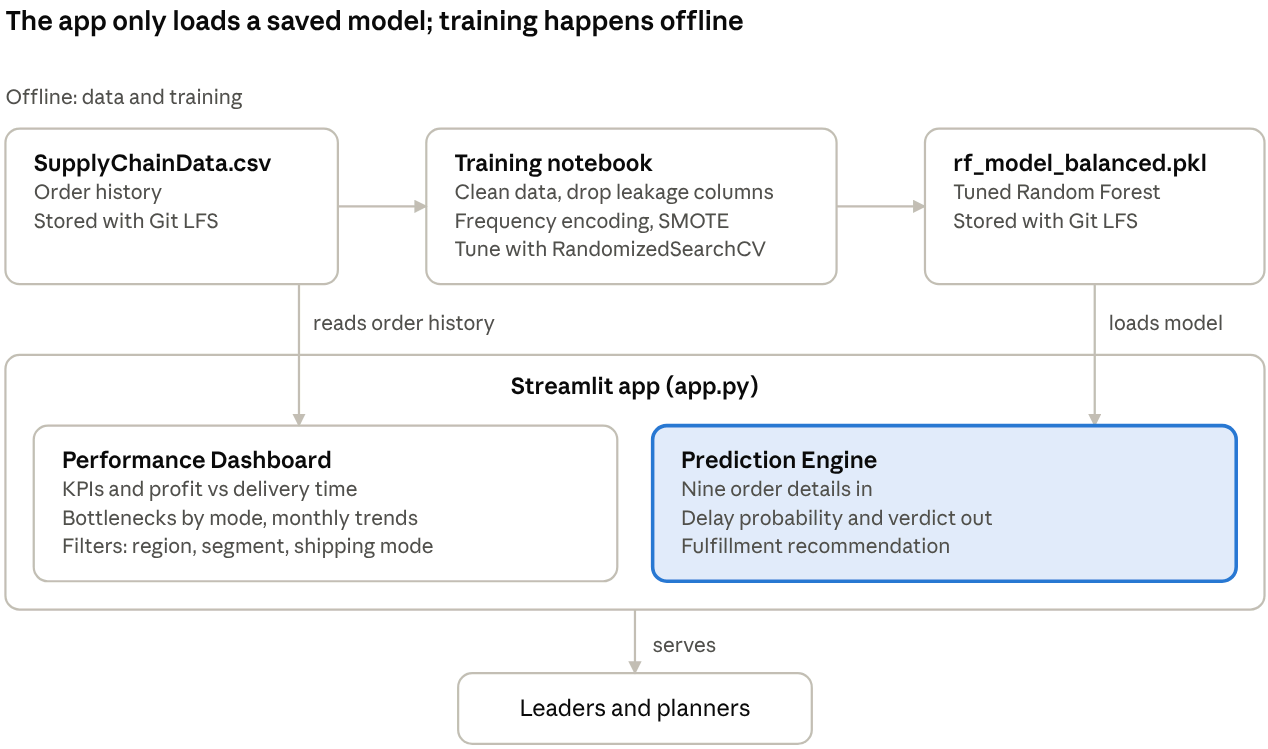
Page	Primary user	Question it answers
Performance Dashboard	Operations and supply chain leaders	Where are we late, how much does it cost and when does it happen?
Prediction Engine	Order planners and fulfillment teams	Is this specific order likely to be late, and what should we do?

Page 1: Performance Dashboard. Three global filters (Order Region, Customer Segment, Shipping Mode) re-slice every section:

- **Key Performance Indicators:** total orders, late deliveries, late and on-time rates, total profit, profit at risk, 90th percentile delay and average order profit.
- **Profitability vs Delivery Time:** a dual-axis chart of total profit (bars) and mean profit (line) by delay in days.
- **Bottleneck Detection:** delay percentage by shipping mode.
- **Time-Based Analysis:** delay percentage by month.
- **Strategic Insights:** plain-language callouts for executives.

Page 2: Prediction Engine. The user enters transaction type, customer segment, region, category, department, shipping mode, scheduled shipping days, order month and order hour. The app returns a delay probability, an on-time or at-risk verdict and a fulfillment recommendation.

Architecture. Data and training happen offline in a notebook. The app only loads the saved model, so it stays light and starts fast.



system architecture · offline training, two-page app

The notebook produces the saved model once; the app serves the dashboard from order history and the prediction page from that model.

7. Key Findings & Business Insights

Shipping mode, not season, is the dominant driver of lateness, and delays are short enough to be fixed by process changes rather than capacity.

Finding	Evidence in the dashboard	Business implication
Lateness is the default	98,743 late orders; 54.70% late rate	Planning assumptions and promises are miscalibrated, not occasionally missed
Shipping mode explains the most	First Class 100%, Second Class 79.7%, Standard Class 39.8% late; Same Day shows no late share	Faster modes miss their promised dates most often; check whether scheduled days are realistic

Finding	Evidence in the dashboard	Business implication
Delays are short	90th percentile delay is 3 days	Extremes are contained, so the cause is systemic and process-driven
No strong seasonality	Monthly late rate stays within about 54.1% to 55.5%	Peak-season capacity is not the main cause; look at daily operations
Profit sits in late orders	$2.1M of profit on delayed orders; the 1-day-late group has the largest profit pool, about $1.2M	Small slips touch the most revenue, so modest improvements recover real money
Long delays cost margin per order	Mean profit per order dips from about $23 at 2 days early to about $19.7 at 3 days late	Late orders erode margin as well as satisfaction

Reading the First Class result. A 100% late rate for the fastest paid tier is either a genuine fulfillment failure or a sign that scheduled days are set tighter than the operation can deliver. Either way it is the first place to investigate, because it combines a premium price with the worst service outcome.

Chart values above are read from the dashboard and are approximate where marked.

8. Industry Relevance & Use Cases

The same pattern, scoring an order at placement and watching delay by mode, region and month, applies wherever a delivery promise is made and measured.

Sector	How the system applies	Standard KPI it supports
E-commerce and retail	Flag at-risk orders at checkout; upgrade mode or reset the promised date before the customer is disappointed	On-time delivery, order cycle time, support contacts per order
Third-party logistics (3PL)	Prioritize pick and pack for high-risk orders; evidence for SLA reviews with clients and carriers	SLA compliance, carrier scorecards

Sector	How the system applies	Standard KPI it supports
Manufacturing and distribution	Anticipate late outbound shipments to key accounts and plan buffer stock	OTIF (on time, in full), perfect order rate
Finance and operations leadership	Size profit exposure from delays and rank fixes by recovered margin	Profit at risk, cost to serve

Why it is practical to adopt.

- **Inputs are available at order time.** No tracking feed or carrier integration is needed to score an order.
- **Output is actionable.** A probability can drive rules such as expedite, split shipment, or proactive customer notice.
- **Outputs map to metrics executives already track,** so the case for investment is stated in OTIF, SLA and margin terms.
- **The stack is light.** Python, scikit-learn and Streamlit deploy on a single small host, which suits pilots before enterprise integration.

Roles it serves. Supply chain analysts use the dashboard; planners use the prediction engine; data teams extend the model; finance uses the profit-at-risk view.

9. Recommendations & Implementation Roadmap

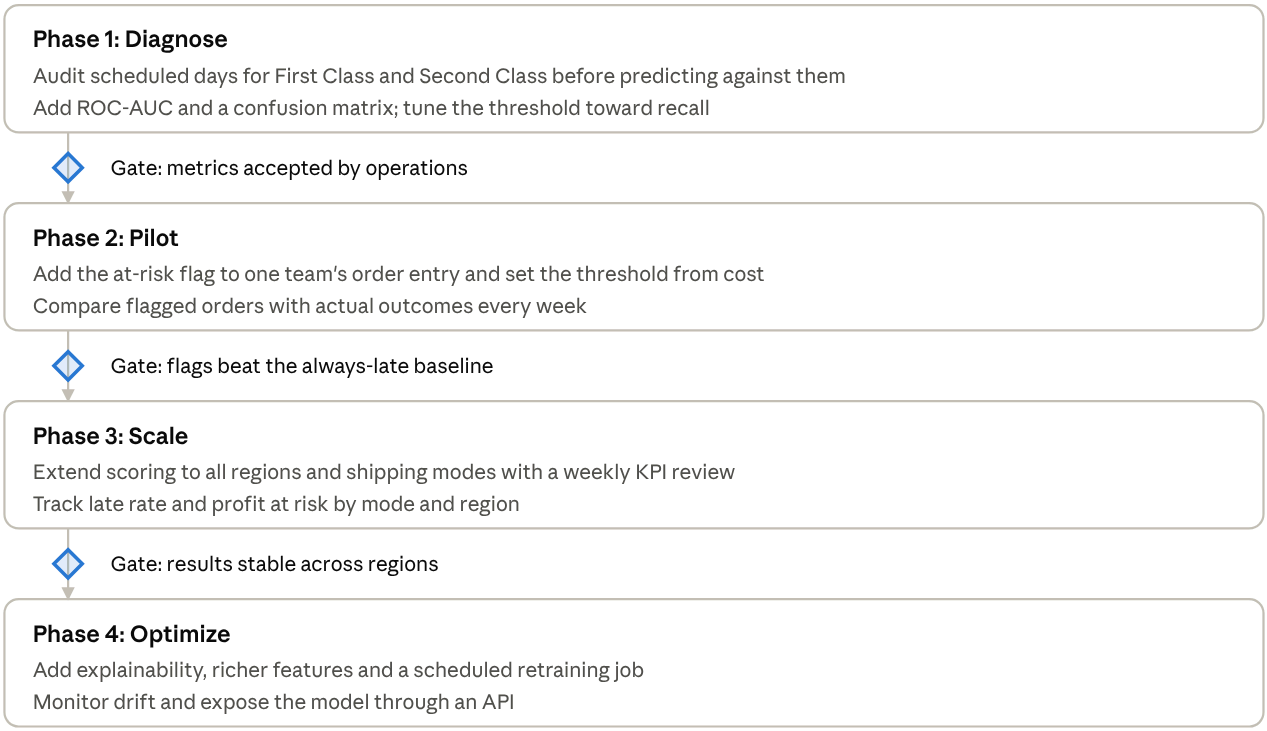
Start by fixing the promise on the fastest shipping modes, then use the model to protect the orders that remain at risk.

#	Recommendation	Rationale from the analysis
1	Audit scheduled shipping days for First Class and Second Class	Late rates of 100% and 79.7% suggest promises tighter than operations can meet
2	Add the at-risk flag to order entry as a pilot	Inputs are known at order time, so action is possible before shipment
3	Set the decision threshold from cost, not default	Expedite cost and missed-delivery cost differ by business

#	Recommendation	Rationale from the analysis
4	Review daily and weekly operations, not seasonal capacity	The monthly late rate is nearly flat across the year
5	Track late rate and profit at risk weekly by mode and region	Keeps the 54.70% baseline and $2.1M exposure visible to owners

The roadmap below is a proposed rollout, not a commitment; durations depend on each organization's data access and change process.

Each rollout phase is released only through a gate



proposed rollout · 4 phases, 3 gates

The pilot has to prove value before scaling, and each later phase starts only when the gate above it is met.

10. Risks, Limitations & Responsible Use

The model is a decision aid trained on one historical dataset, and it needs monitoring before anyone relies on it for customer-facing promises.

Risk	What could go wrong	Mitigation
Missed late orders	About one in four late orders is missed, and about one in five flagged orders is a false alarm	Add ROC-AUC, a confusion matrix and cross-validation; tune the threshold toward recall
Synthetic training data	SMOTE creates artificial minority samples that can overfit	Apply it to the training split only; evaluate on untouched real orders
Data drift	Carrier performance, product mix and processes change over time	Retrain on a schedule; monitor input and outcome distributions
Single-source data	One dataset may not represent other regions or business models	Validate on the target organization's own history before use
Hidden leakage	A feature correlated with the outcome could slip in	Review each feature for availability at order time
Over-trust in a probability	Planners may treat 48% and 52% as different realities	Show confidence bands and recommend actions by risk tier
Artifact size and reproducibility	Large files need Git LFS; a model without its encoders cannot score correctly	Version the model together with its encoding maps and package versions

Responsible use. The model predicts operational risk for orders, not behavior of individuals. It should support planners, not automatically penalize customers, staff or carriers.

11. Future Enhancements

The highest-value next steps make the model explainable, measurable in production and connected to live operations.

- **Explainability:** add SHAP or permutation importance so planners see why an order is flagged, and so the First Class result can be traced to its drivers.
- **Fuller evaluation:** cross-validation, a precision-recall curve and cost-based threshold selection, published in the app.

- **Model comparison:** benchmark against gradient boosting such as XGBoost or LightGBM and a calibrated logistic regression.
- **Richer features:** carrier, warehouse, distance, inventory level, weather and holiday calendars.
- **Delay-length prediction:** extend from late or on time to expected days late, which supports tiered customer messages.
- **MLOps:** a scheduled retraining job, model registry, automated tests and drift alerts.
- **Integration:** serve the model through an API so order management and warehouse systems can call it directly.
- **Batch scoring:** upload a file of open orders and receive a ranked list by risk.

12. Technology Stack & Reproducibility

The project runs from a single repository and can be rebuilt on any machine with Python, Git and Git LFS.

Layer	Tools
Language and data	Python, Pandas, NumPy
Modeling	Scikit-learn Random Forest, RandomizedSearchCV
Class balancing	Imbalanced-learn (SMOTE)
Visualization	Plotly
Application	Streamlit (two pages, sidebar navigation)
Packaging	`rf_model_balanced.pkl`, `requirements.txt`, virtual environment `v1`
Version control	Git, with Git LFS for the CSV and model file

Repository layout.

```
Raw_Data/                          SupplyChainData.csv (Git LFS)
Data_Modeling_&_Model_Training/    Jupyter notebook
Prediction_App_&_Dashboard/        app.py, requirements.txt,
rf_model_balanced.pkl (Git LFS)
images/                            Dashboard screenshots
```

Run it. Clone the repository, run `git lfs pull`, create the `v1` virtual environment, install `requirements.txt` from the app folder and start the app with `streamlit run app.py`. A hosted version is live on Streamlit Community Cloud, and the full source is on GitHub.

13. Conclusion

The Supply Chain Intelligence System shows that a 54.70% late rate and $2.1M of exposed profit can be both explained and acted on. The analysis points to shipping mode and process, not season or extreme outliers, as the places to look first. The prediction engine adds an early warning at order placement with 74% accuracy, built on leakage-free features and a balanced training set.

The next step toward production is to add ROC-AUC and a confusion matrix, set the decision threshold from cost, and pilot the at-risk flag with a single team. That turns a portfolio-grade analysis into an operating tool.